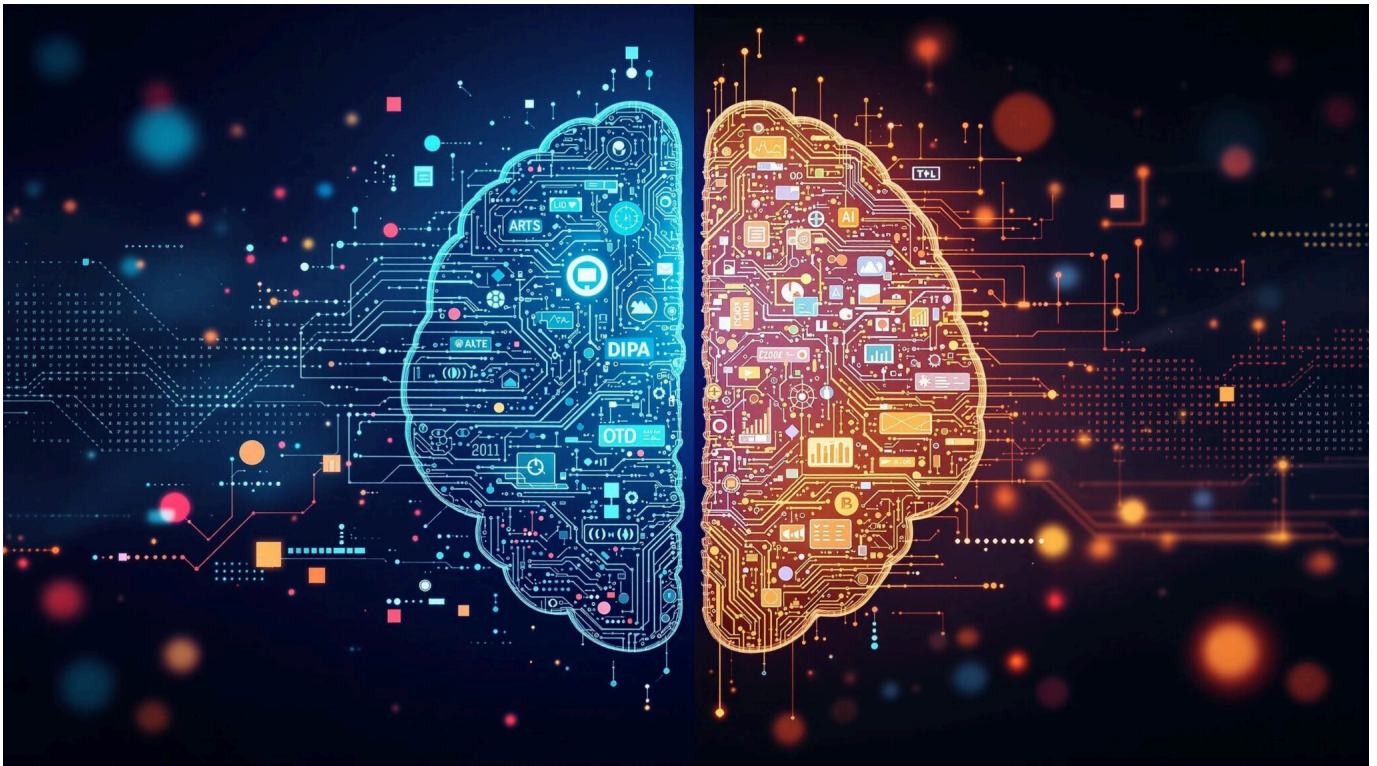


Trabajo de Clasificación binaria

MACHINE LEARNING

Profesorado: Inmaculada Gutiérrez, Daniel Gómez, Javier Castro



Alumna: Oumaima Ech Chdig

UNIVERSIDAD
COMPLUTENSE
DE MADRID



1.Introducción y Objetivo

El presente trabajo presenta un sistema de clasificación con **aprendizaje supervisado** para la industria de las telecomunicaciones, con el objetivo de prever el abandono de clientes (Churn), representado por la variable dependiente Y.

La habilidad de prever qué clientes tienen una alta **probabilidad de darse de baja (Y=1)** es una ventaja competitiva fundamental. Esto permite a la empresa maximizar la utilización de recursos comerciales y reducir la pérdida de ingresos. Para asegurar la solidez de la predicción, se evaluará el desempeño de modelos de **Máquinas de Vectores de Soporte (SVM)**, haciendo una comparación con técnicas de ensamblaje de alta precisión como **Bagging** y **Stacking**.

2. Descripción y Análisis de los Datos

El conjunto de datos entregado incluye 20 variables independientes (V1 a V20) que contienen información sobre demografía, planes contratados y detalles sobre el consumo.

Las variables se pueden clasificar en cuatro categorías principales:

- **Identificación y Ubicación:** V1 (Estado), V3 (Código de área) y V4 (Número telefónico).
- **Planes y Servicios:** V5 (Plan internacional), V6 (Buzón de voz) y V7 (Mensajes de voz).
- **Métricas de Uso** (Minutos y Llamadas): Se dividen por horarios (diurnos, vespertinos, nocturnos e internacionales) usando las variables V8 a V19.
- **Atención al Cliente:** V20, que contabiliza el total de llamadas al soporte técnico.

3. Análisis y Depuración de datos

Primero se configura el espacio de trabajo necesario para realizar el **análisis de datos**, la **visualización** y el **modelo** posterior. Para esto, se traen las bibliotecas claves (pandas, numpy, seaborn, sklearn) y se abre el conjunto de datos BBDD_ML_TAREA. Csv.

Código:

```
df = pd.read_csv('BBDD_ML_TAREA.csv')
```

	V1	V2	V3	V4	V5	V6	V7
1	43	121	415	1118	0	0	0
2	7	170	510	3326	0	0	0
3	31	96	510	2146	0	0	0
4	17	90	415	387	0	0	0
5	7	61	415	313	0	0	0
6	45	104	415	3280	0	0	0
7	32	93	415	380	0	0	0
8	5	88	510	2384	0	0	0
9	15	81	415	3186	0	0	0
10	49	130	510	374	0	0	0
11	34	99	415	3068	0	0	0
12	48	184	408	3873	0	1	12
13	15	105	510	526	0	1	42
14	0	173	510	1167	0	0	0
15	9	31	510	3921	0	0	0
16	29	161	415	69	0	0	0
17	49	95	415	1553	0	0	0
18	19	132	408	750	0	1	36
19	7	93	408	4156	0	0	0
20	2	89	415	1981	0	1	25

A continuación se realiza un análisis inicial del dataset .

Código:

df.head()

df.dtypes

```
Número de filas: 9200, Número de columnas:21
V1      int64
V2      int64
V3     float64
V4      int64
V5      int64
V6      int64
V7      int64
V8     float64
V9      int64
V10     float64
V11     float64
V12     int64
V13     float64
V14     float64
V15     float64
V16     float64
V17     float64
V18     int64
V19     float64
V20     int64
Y       int64
dtype: object
```

El conjunto de datos tiene 9,200 registros y una estructura adecuada, donde las 21 variables (incluyendo la variable dependiente Y) son **numéricas**. Esta uniformidad, junto con el hecho de que variables como V5 y V6 ya están convertidas de texto ("sí"/"no") a números (0/1), facilita no necesitar transformaciones extra como el **One-Hot Encoding**.

La primera medida consiste en eliminar la variable V4 (Número de teléfono) ya que es un identificador único que **no aporta valor estadístico**. Su eliminación es clave para facilitar el modelo y prevenir el **ajuste excesivo**, evitando que el algoritmo memorice casos específicos en vez de reconocer patrones generales de abandono.

Código:

```
X = df.drop(['V4'], axis=1)
```

● Gestión de Valores Ausentes

Después de la revisión inicial, se llevó a cabo la identificación de la existencia de **valores ausentes** en el conjunto de datos. La falta de información puede distorsionar los resultados o dificultar la aplicación de los algoritmos de aprendizaje.

Código:

```
print("Valores nulos por columna:")
```

```
print(df.isnull().sum())
```

Se encontraron datos faltantes en cinco variables (**V3, V10, V14, V15 y V19**), con una frecuencia muy baja en comparación con un total de 9,200 entradas. Para mantener la calidad del conjunto de datos y evitar la pérdida de información importante al quitar filas, se decidió usar la **mediana** para rellenar los valores.

Valores nulos por columna:	valores nulos por columna:
V1 0	V1 0
V2 0	V2 0
V3 75	V3 0
V4 0	V4 0
V5 0	V5 0
V6 0	V6 0
V7 0	V7 0
V8 0	V8 0
V9 0	V9 0
V10 26	V10 0
V11 0	V11 0
V12 0	V12 0
V13 0	V13 0
V14 72	V14 0
V15 60	V15 0
V16 0	V16 0
V17 0	V17 0
V18 0	V18 0
V19 44	V19 0
V20 0	V20 0
Y 0	Y 0
dtype: int64	dtype: int64

Se optó por la mediana en lugar de la media, ya que la mediana es una medida de **tendencia central** que es más resistente a los valores anómalos.

Código:

```
columnas_con_nulos = ['V3', 'V10', 'V14', 'V15', 'V19']
```

```

for col in columnas_con_nulos:

df[col] = df[col].fillna(df[col].median())

print(df.isnull().sum())

```

- **Análisis de Categorías y Relaciones (Tablas de Contingencia)**

Se han examinado las variables categóricas a través de tablas de contingencia con el fin de reconocer patrones de conducta anteriores al entrenamiento.

Código:

```

variables_clave = ['V5', 'V6', 'V20', 'V3']
for var in variables_clave:
print(f"\nTabla de Contingencia: {var} vs Abandono (Y)")
tabla = pd.crosstab(df[var], df['Y'], normalize='index')
print(tabla)
print("-" * 50)

```

```

Tabla de Contingencia: V5 vs Abandono (Y)
Y      0      1
V5
0  0.563991  0.436009
1  0.199008  0.800992

```

```

Tabla de Contingencia: V6 vs Abandono (Y)
Y      0      1
V6
0  0.455670  0.544330
1  0.659182  0.340818

```

```

Tabla de Contingencia: V20 vs Abandono (Y)
Y      0      1
V20
0  0.553371  0.446629
1  0.587636  0.412364
2  0.424667  0.575333

```

```

Tabla de Contingencia: V3 vs Abandono (Y)
Y      0      1
V3
408.0  0.488106  0.511894
415.0  0.504961  0.495039
510.0  0.501744  0.498256

```

Después de realizar el **análisis de contingencia**, se reconocen V5, V6 y V20 como las variables que más afectan el abandono. Es importante mencionar el caso de V5 (Plan Internacional), en el que el riesgo de que se vayan se duplica al contratar el servicio, y V20 (Atención al Cliente), donde se observa un límite crítico de abandono a partir de la segunda llamada. Estos resultados confirman la relevancia de estas variables para la capacidad de predicción del modelo final.

- **Recategorización de Variables**

Se implementó un método de agrupamiento (**binning**) sobre la variable V20 (Llamadas de soporte), uniendo los valores que son 2 o más en una sola categoría. Esta elección se basa en la escasa representación estadística de los valores altos (por ejemplo, solo hay 6 casos para el valor '8'), cuya inclusión podría generar ruido y aumentar el riesgo de sobreajuste. De esta forma, el modelo SVM es capaz de detectar de manera más efectiva cómo influye la insistencia del cliente en la posibilidad de abandono.

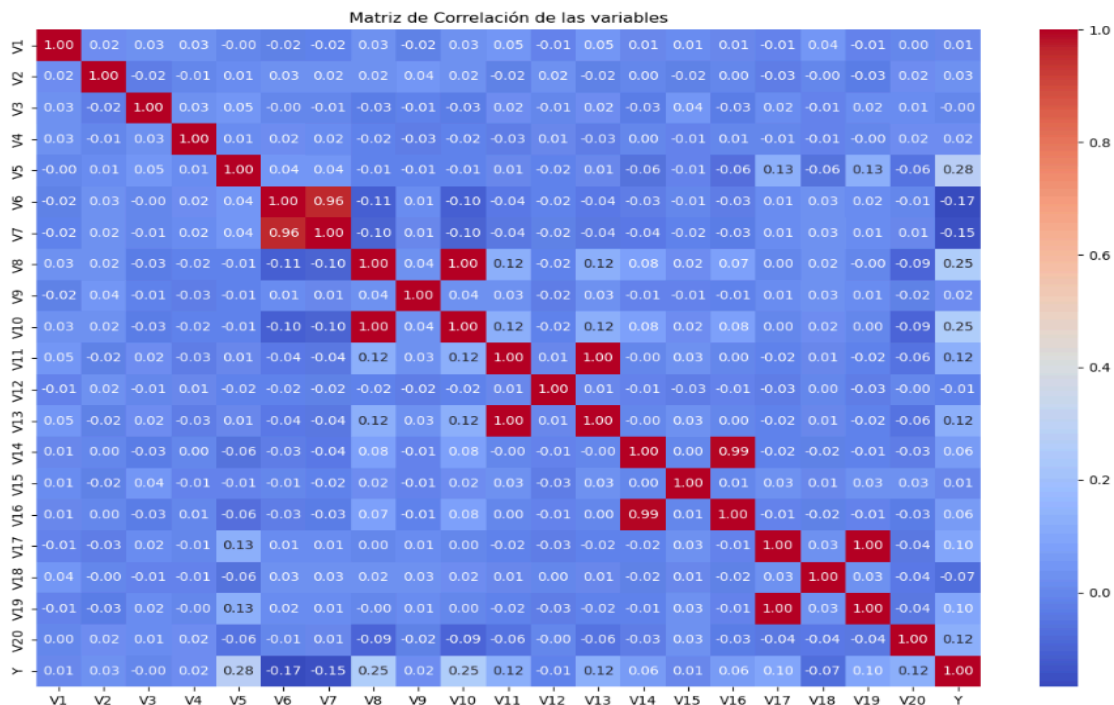
```
print(df["V20"].value_counts())
```

```
Frecuencias de V20:
V20
1    2847
2    1860
0    1780
3    1129
4     924
5     447
6     141
7      47
9      19
8       6
Name: count, dtype: int64
```

```
df["V20"] = df["V20"].apply(lambda x: 2 if x >= 2 else x)
```

- **Gestión de la Multicolinealidad**

A continuación realizamos un análisis de las relaciones entre variables, calculando la matriz de correlación:



Código:

```
plt.figure(figsize=(15, 10))

sns.heatmap(df.corr(), annot=True, fmt=".2f", cmap='coolwarm')

plt.title("Matriz de Correlación de las variables") plt.show()
```

Se quitaron las variables de costo (V10, V13, V16 y V19) por tener una correlación perfecta ($r=1.0$) con las de minutos. Al ser métricas repetidas, su eliminación evita problemas de **multicolinealidad** y hace el modelo más simple **sin perder información**, puesto que el patrón de uso ya se encuentra en las variables de minutos.

Código:

```
variables_redundantes = ['V7', 'V10', 'V13', 'V16', 'V19']

df.drop(columns=variables_redundantes, inplace=True)
```

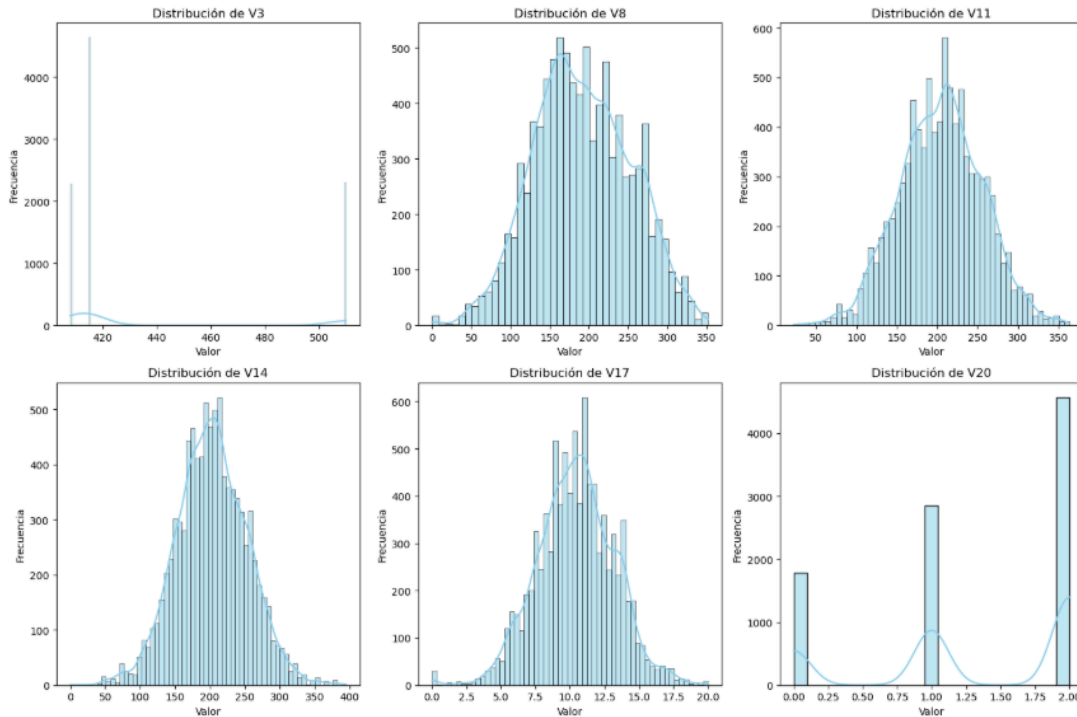
● Gráficos

A continuación realizamos los histogramas de algunas variables significativas:

Código:

```
variables_analisis = ['V3', 'V8', 'V11', 'V14', 'V17', 'V20']

plt.figure(figsize=(15, 10))
for i, col in enumerate(variables_analisis):
    plt.subplot(2, 3, i+1)
    sns.histplot(df[col], kde=True, color='skyblue')
    plt.title(f'Distribución de {col}')
    plt.xlabel('Valor')
    plt.ylabel('Frecuencia')
plt.tight_layout()
plt.show()
```



El análisis a través de histogramas facilitó la organización de los predictores en tres categorías:

- **Variables de Consumo (V8, V11, V14):** Muestran una **Distribución Normal** casi perfecta. Esta simetría resulta ideal para el modelado, aunque la discrepancia de escalas con V17 (0-20 frente a 0-350) hace necesario el uso de **StandardScaler** para prevenir sesgos debidos a la magnitud.
- **Variable Geográfica (V3):** Su perfil multimodal confirma que es categórica (códigos de área), funcionando más como un indicador de **región** que como una cantidad.
- **Variable de Comportamiento (V20):** Exhibe un **sesgo positivo** considerable, concentrando la mayor parte de los registros en valores bajos. Este comportamiento justifica la recategorización ($x \geq 2$) que se llevó a cabo para estabilizar la **varianza** y aumentar la **robustez** de la SVM.
- **División del Dataset (Train/Test)**

Se ha separado el conjunto de datos en dos partes: **entrenamiento (80%)** y **prueba (20%)**, empleando una semilla constante para garantizar que los experimentos puedan repetirse. Se ha implementado la estratificación (**stratify=y**), que es una técnica esencial para conservar la misma proporción de la variable objetivo en los dos grupos. Esto ayuda a evitar sesgos y permite que el modelo se evalúe con un conjunto representativo de la realidad, lo que posibilita medir su desempeño real ante datos no conocidos.

Código:

```
# Quitamos 'Y' porque es lo que queremos predecir
```

```
X = df.drop(columns=["Y"])
```

```
y = df['Y']
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=12345, stratify=y)
```

- **Estandarización de los Datos (StandardScaler)**

Se ha realizado una **transformación** para que todas las variables tengan una **media de 0** y una **varianza de 1**. Debido a que las SVM fundamentan su aprendizaje en el cálculo de distancias entre puntos, es esencial que todas las dimensiones tengan la misma escala.

Código:

```
scaler = StandardScaler()
```

```
X_train_sc = scaler.fit_transform(X_train)
```

```
X_test_sc = scaler.transform(X_test)
```

Comprobamos el equilibrio de clases:

```
print(df['Y'].value_counts(normalize=True))
```

```
Y
0    0.5
1    0.5
Name: proportion, dtype: float64
```

El conjunto de datos muestra una **distribución perfecta** entre las dos categorías (50% bajas y 50% permanencias), lo que permite al modelo entrenarse sin prejuicios hacia la clase más numerosa. Esta igualdad posibilita la utilización del **Accuracy** como la principal métrica de desempeño, eliminando la necesidad de técnicas de equilibrio como el **SMOTE**. En este contexto, se ha dado prioridad al uso de SVM que se muestra más resistente al fundamentar su clasificación en la maximización del margen a través de **hiperplanos**.

3. Modelado con Máquina de Vectores de Soporte (SVM)

En esta sección, se llevará a cabo el entrenamiento de modelos de **Máquinas de Soporte Vectorial (SVM)**, utilizando diferentes tipos de **kernels**, con la finalidad de captar de manera efectiva tanto las relaciones **lineales** como las **no lineales** que se encuentran en el conjunto de datos. Para garantizar la mayor precisión posible, se llevará a cabo una optimización de los **hiperparámetros** a través de una búsqueda completa (**Grid Search**). El proceso de selección se basará en una evaluación estricta del rendimiento utilizando las métricas de **Precisión** y **AUC**, además de incluir una comparación visual de las **curvas ROC** que ayude a identificar y justificar de manera técnica la configuración más efectiva.

- **Kernel Lineal**

El Kernel Lineal funciona utilizando el **producto de los vectores** de características iniciales, conservando la misma dimensión de los datos sin implementar transformaciones no lineales.

Para optimizarlo, se estableció un rango de búsqueda para el parámetro **C** que va de 0. 1 a 100, dando más importancia a una mayor concentración en el intervalo [0. 1, 1]. Esta técnica permite ajustar con precisión **la tolerancia al error**, lo que ayuda a encontrar un margen de decisión sólido que **elimine el ruido estadístico** y favorezca patrones generales frente a casos únicos.

Además, utilizar valores más elevados (hasta 100) ayuda a evaluar si una penalización rigurosa del error no incrementa la precisión, lo que sugiere que el modelo ha llegado a **su máximo en el aprendizaje lineal**, lo que justifica la transición hacia kernels más avanzados.

Código:

```
print("Iniciando búsqueda para Kernel Lineal...")
param_grid_lineal = {'C':[0.1, 0.2, 0.3, 0.4, 0.5, 1, 2, 5, 10, 100]}
skf = StratifiedKFold(n_splits=4, shuffle=True, random_state=12345)
grid_lin = GridSearchCV(SVC(kernel='linear', probability=True),
                        param_grid_lineal,
                        refit=True,
                        cv=skf,
                        verbose=3)

predictions_lin = grid_lin.predict(X_test_sc)
print(classification_report(y_test, predictions_lin))

Iniciando búsqueda para Kernel Lineal...
Fitting 4 folds for each of 10 candidates, totalling 40 fits
[CV 1/4] END .....C=0.1;; score=0.701 total time= 8.0s
[CV 2/4] END .....C=0.1;; score=0.711 total time= 9.6s
[CV 2/4] END .....C=100;; score=0.711 total time= 4.1min
.....
[CV 4/4] END .....C=100;; score=0.724 total time= 4.1min
```

Se examinó el parámetro C usando **Grid Search (CV=4)**. Los resultados indican un **Accuracy** mantenido en aproximadamente **71%**, sin cambios importantes al incrementar la penalización.

Es necesario utilizar kernels **no lineales** para captar **relaciones más complejas**.

A continuación se realiza la gráfica de precisión y el diagrama de cajas.

Código:

```
# Entrenamos (Usando tus variables del Punto 1)
resultados_lin = grid_lin.fit(X_train_sc, y_train)
resultados_df_lin = pd.DataFrame(resultados_lin.cv_results_)
plt.figure(figsize=(8, 4))
plt.scatter(resultados_df_lin['param_C'], resultados_df_lin['mean_test_score'], color='b')
```

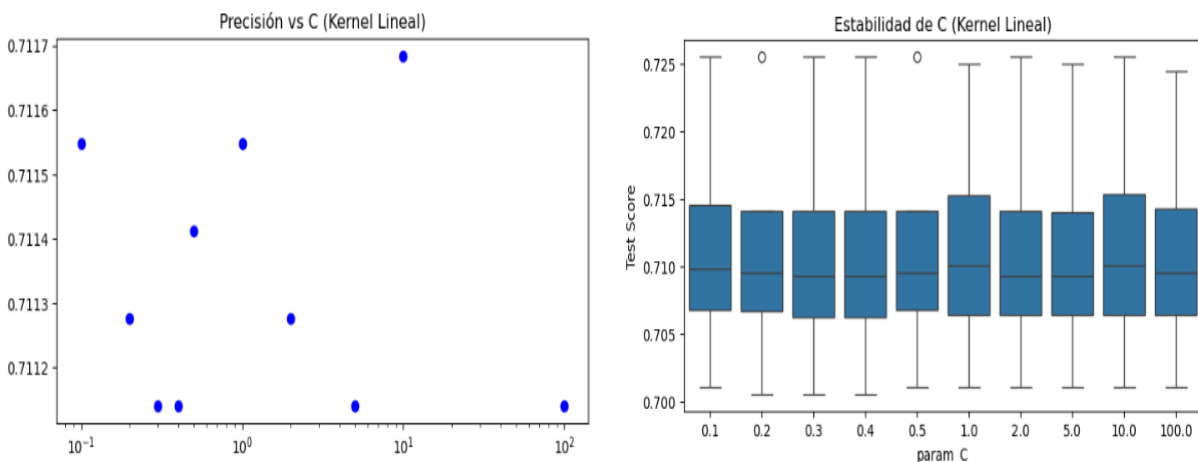
```

plt.xscale('log')
plt.title('Precisión vs C (Kernel Lineal)')
plt.show()

# Boxplot de estabilidad
scores_lin_df = pd.melt(resultados_df_lin, id_vars=['param_C'],
                        value_vars=[f'split{i}_test_score' for i in range(4)],
                        var_name='Split', value_name='Test Score')
plt.figure(figsize=(8, 4))
sns.boxplot(x='param_C', y='Test Score', data=scores_lin_df)
plt.title('Estabilidad de C (Kernel Lineal)')
plt.show()

# Métricas finales
predictions_lin = grid_lin.predict(X_test_sc)
print(classification_report(y_test, predictions_lin))

```



La **gráfica de precisión** indica una variación mínima (0.1) entre los diferentes valores de C. Esto respalda que el modelo lineal ha llegado a su **máximo en capacidad de predicción**, justificando la elección de investigar kernels no lineales para mejorar los resultados.

El **gráfico de cajas** respalda la consistencia del kernel lineal, con medianas de precisión cerca del 71%. Se evidencia que el modelo es sólido y no muestra sobreajuste.

A continuación observamos cual es el mejor estimador:

`grid_lin.best_estimator_`

```
SVC(C=10, kernel='linear', probability=True)
```

Después de elegir el hiperparámetro más adecuado (C=10), se creó la **matriz de confusión** para analizar cómo se desempeñó el modelo en el conjunto de pruebas. *Código:*

```

model_final = SVC(kernel='linear', C=10, probability=True)
model_final.fit(X_train_sc, y_train)
# Predicciones finales
predictions = model_final.predict(X_test_sc)
print(classification_report(y_test, predictions))
cm = confusion_matrix(y_test, predictions)
print("Matriz de Confusión:\n", cm)
accuracy = (cm[0,0] + cm[1,1]) / cm.sum()
print(f'Accuracy Final: {accuracy:.4f}')

              precision    recall  f1-score   support

      0         0.69       0.76       0.73         920
      1         0.74       0.66       0.70         920

 accuracy          0.71
 macro avg         0.71
weighted avg         0.71

Matriz de Confusión:
[[701 219]
 [312 608]]
Accuracy Final: 0.7114

```

El modelo final consigue una precisión del 71.14%. El análisis de la matriz de confusión muestra una habilidad predictiva balanceada, logrando **clasificar con éxito a 1,309** clientes (701 + 608); sin embargo, se observa un espacio para mejorar en los **312 falsos negativos**. Además, el modelo evidencia solidez y una adecuada generalización con un **F1-score** equilibrado de **0.71**.

- **Kernel RBF**

En contraste con el método lineal, el **Kernel de Función de Base Radial (RBF)** transforma los datos originales a un espacio con características de **dimensión infinita**, lo que facilita la captura de límites de decisión muy complejos. Este tipo de kernel mide la relación entre observaciones según su **cercanía**; los puntos que están más cerca reciben una alta puntuación de similitud, mientras que la influencia disminuye de forma exponencial a medida que se incrementa la distancia en el espacio de entrada.

Para mejorar el modelo, se llevó a cabo un proceso de **búsqueda en rejilla** (Grid Search) sobre el hiperparámetro **gamma** con valores que oscilan entre 0.001 y 1. Este parámetro es fundamental, ya que regula el **alcance de influencia** de cada punto de entrenamiento: valores bajos hacen que la frontera de decisión sea más suave, mientras que valores altos permiten una adaptación más ajustada a la forma de los datos. El proceso de entrenamiento se realizó usando un método de **Validación Cruzada Estratificada (StratifiedKfold)**.

```

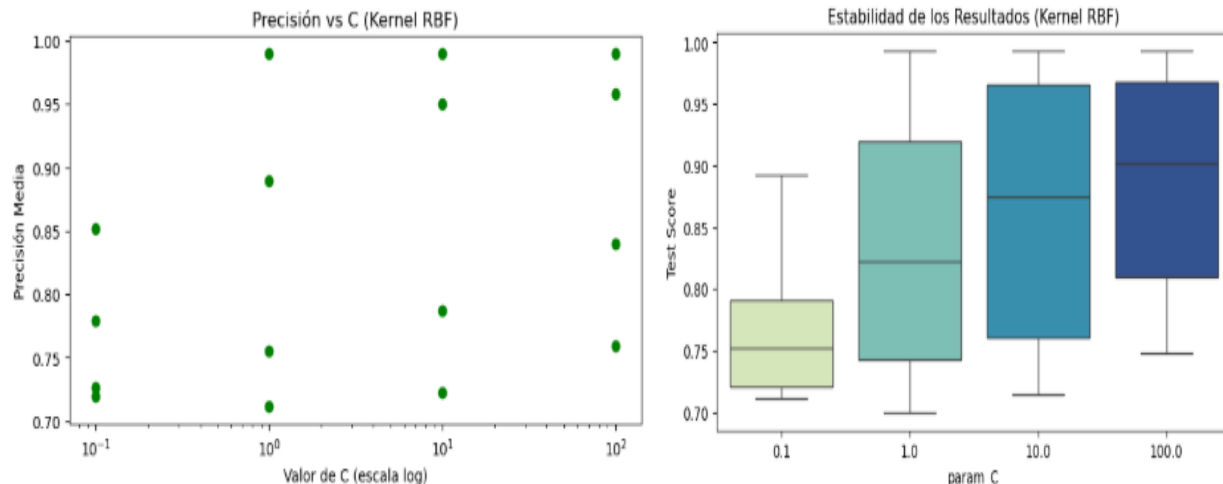
Iniciando búsqueda para Kernel RBF...
Fitting 4 folds for each of 16 candidates, totalling 64 fits
[CV 1/4] END .....C=0.1, gamma=0.001;, score=0.715 total time= 18.4s
[CV 2/4] END .....C=0.1, gamma=0.001;, score=0.711 total time= 16.9s
.....

```

```
[CV 3/4] END .....C=100, gamma=1;; score=0.986 total time= 11.3s
[CV 4/4] END .....C=100, gamma=1;; score=0.990 total time= 10.3s
```

Al implementar, notamos que en la transición al Kernel RBF el modelo consigue una precisión de aproximadamente **99%** al identificar patrones complicados que antes no se podían ver. La configuración de **gamma=1** fue clave, consiguiendo una frontera de decisión más exacta.

Realizamos la gráfica de precisión y el diagrama de cajas (el código a realizar es el mismo que el anterior):



Al hacer **la gráfica de precisión**, notamos que, a diferencia del modelo lineal, se observan **bandas** claras de precisión, lo que muestra que tiene **mejor capacidad de predicción**.

Los **diagramas de caja** del Kernel RBF indican un **aumento en la precisión** a medida que **se incrementa C**. El tamaño de las cajas muestra cómo se relaciona con el parámetro gamma, logrando medianas mucho más altas que las del modelo lineal.

Los mejores parámetros del modelo, obtenido por la función `best_params_` son:

```
{'C': 1, 'gamma': 1}
```

La matriz de confusión en este caso obtenemos:

```
[[918  2]
 [  6 914]]
accuracy 0.9956521739130435
```

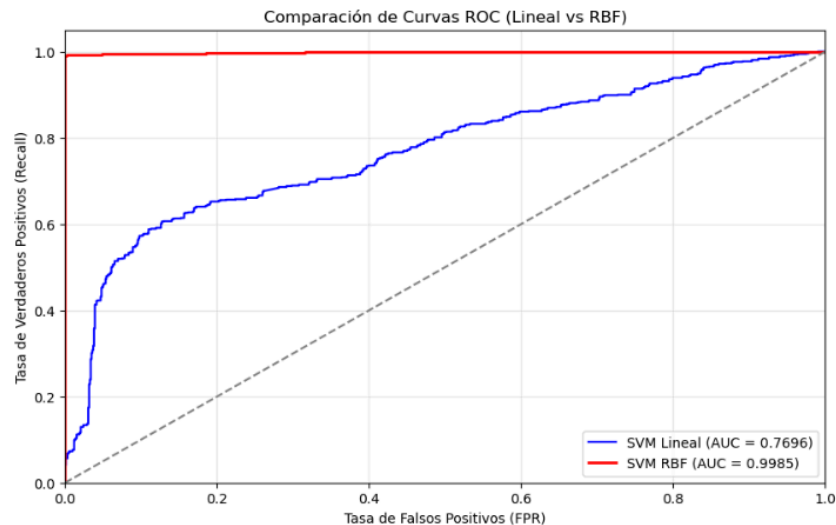
El modelo RBF obtiene una **precisión del 99.57%**, identificando correctamente a **1,832** de **1,840** clientes. Con un error total de solo 0.43%, la matriz de confusión valida la eficacia de este kernel para entender la lógica del conjunto de datos. La casi inexistente cantidad de **falsos negativos** (6 casos) lo convierte en una herramienta precisa para retener clientes.

● Comparativa

Para realizar la comparación, se determinaron las probabilidades de pertenecer a cada categoría utilizando `predict_proba`. La estimación del **AUC (Área Bajo la Curva)** ofrece una medida general de desempeño. [Código:](#)

1. Obtenemos las probabilidades de los mejores modelos encontrados por GridSearch

```
y_score_lin = grid_lin.predict_proba(X_test_sc)[: , 1]
y_score_rbf = grid_rbf.predict_proba(X_test_sc)[: , 1]
fpr_lin, tpr_lin, _ = roc_curve(y_test, y_score_lin)
roc_auc_lin = auc(fpr_lin, tpr_lin)
fpr_rbf, tpr_rbf, _ = roc_curve(y_test, y_score_rbf)
roc_auc_rbf = auc(fpr_rbf, tpr_rbf)
```



El modelo lineal muestra una línea más recta, acercándose a la diagonal y la del RBF se adhiere casi totalmente a la parte superior izquierda.

Entrenamos el ganador final (RBF) con sus mejores parámetros:

```
final_model = SVC(kernel='rbf', C=grid_rbf.best_params_['C'],
                  gamma=grid_rbf.best_params_['gamma'], probability=True)
final_model.fit(X_train_sc, y_train)
final_preds = final_model.predict(X_test_sc)
print(f"Accuracy final en Test: {accuracy_score(y_test, final_preds):.4f}")
Accuracy final en Test: 0.9957
```

4. Ensamblado: Bagging y Stacking

• Bagging

Bagging es un enfoque de **meta-algoritmo** que busca crear **combinaciones** de diferentes modelos partiendo de un **grupo inicial**, lo que ayuda a reducir la varianza y previene el sobreajuste. Este método se puede aplicar utilizando cualquier tipo de algoritmo como base; en este contexto, el algoritmo base será el resultado de la sección anterior, **SVM con RBF**.

Código:

```
mejor_modelo_base = grid_rbf.best_estimator_  
# 1. Creamos el esquema de Bagging
```

```
bagging_final = BaggingClassifier(  
    estimator=mejor_modelo_base,  
    n_estimators=500,  
    max_samples=0.8,  
    max_features=1.0,  
    bootstrap=True,  
    random_state=12345, # Tu semilla fija [cite: 17, 81]  
    n_jobs=-1  
# 2. Entrenamos el ensamblado  
bagging_final.fit(X_train_sc, y_train)
```

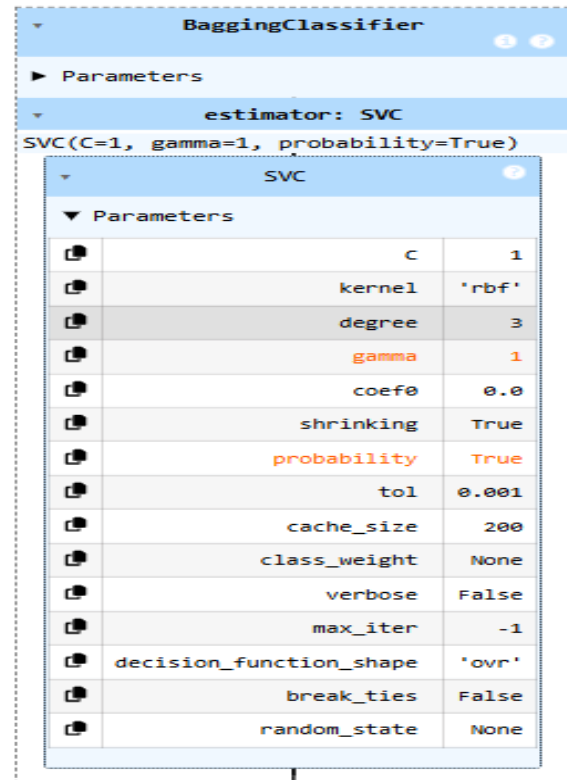
Se han establecido **500 evaluadores independientes**. Cada evaluador se capacita utilizando el **80% de las muestras** del conjunto de datos original elegidas a través de **muestreo con reposición**. Este método garantiza que cada modelo experimente una versión un poco diferente de la realidad, lo que **incrementa la resistencia a fallos**.

La configuración del modelo final se fundamenta en la incorporación de un **BaggingClassifier** que emplea como modelo base una Máquina de Soporte Vectorial.

Se destaca la utilización del **Kernel RBF** con un valor de **C=1** y un **gamma=1**. Esta combinación asegura que cada uno de los modelos en ensamblaje presente una elevada habilidad para discriminar .

El parámetro **probability=True** se encuentra habilitado, lo que permite que el grupo de modelos no solo elija por una clase, sino que también calcule las probabilidades de éxito.

Evaluamos el modelo bagging. *Código:*



```
from sklearn.metrics import classification_report
```

```
print('Bagging - Test Score: %.3f' % bagging_final.score(X_test_sc, y_test))
```

```
print('Bagging - Training Score: %.3f' % bagging_final.score(X_train_sc, y_train))
```

```

y_pred_bagging = bagging_final.predict(X_test_sc)

print(classification_report(y_test, y_pred_bagging))

Bagging - Test Score: 0.996
Bagging - Training Score: 1.000

```

	precision	recall	f1-score	support
0	0.99	1.00	1.00	920
1	1.00	0.99	1.00	920
accuracy			1.00	1840
macro avg	1.00	1.00	1.00	1840
weighted avg	1.00	1.00	1.00	1840

El modelo obtiene una puntuación de entrenamiento (**Training Score**) de 1.000 y una puntuación de prueba (**Test Score**) de 0.996. La pequeña diferencia entre ambos valores (menos de 0.4%) muestra que el Bagging ha funcionado como se esperaba: **ha reducido el riesgo de sobreajuste**. Alcanza un **recall** perfecto (1.00), lo que implica que el modelo reconoce al 100% de los clientes que permanecen. El valor de **F1-Score** es 1.00 en todas las métricas, lo que indica que el sistema es muy sólido.

• Stacking

A diferencia de las técnicas de votación básicas, el Stacking crea un "**meta-modelo**" que **combina** de manera inteligente los resultados obtenidos. Su principal beneficio radica en que puede ajustar la importancia de las predicciones, utilizando las virtudes de cada modelo base (SVM, Árbol, KNN) para rectificar sus fallos individuales y mejorar la precisión total.

```

# 1. Definición de los modelos base
modelos_base = [
    ('svm_rbf', grid_rbf.best_estimator_),
    ('arbol', DecisionTreeClassifier(random_state=12345)),
    ('knn', KNeighborsClassifier(n_neighbors=5))
]

# 2. Definición del meta-modelo
meta_modelo = LogisticRegression(max_iter=1000, random_state=12345)

# 3. Construcción del Stacking
stacking_model = StackingClassifier(
    estimators=modelos_base,
    final_estimator=meta_modelo,
    cv=5,
    n_jobs=-1,
    passthrough=True
)

# 4. Búsqueda paramétrica
param_grid_stack = {
    'final_estimator__C': [0.1, 1.0, 10.0], 'passthrough': [True, False]
}

```

```

grid_stack = GridSearchCV(
    estimator=stacking_model,
    param_grid=param_grid_stack,
    cv=skf,
    scoring='accuracy')
grid_stack.fit(X_train_sc, y_train)

best_estimator_ : StackingClassifier

```

StratifiedKFold

svm_rbf			arbol			knn		
SVC			DecisionTreeClassifier			KNeighborsClassifier		
Parameters			Parameters			Parameters		
C		1	criterion		'gini'	n_neighbors		5
kernel		'rbf'	splitter		'best'	weights		'uniform'
degree		3	max_depth		None	algorithm		'auto'
gamma		1	min_samples_split		2	leaf_size		30
coef0		0.0	min_samples_leaf		1	p		2
shrinking		True	min_weight_fraction_leaf		0.0	metric		'minkowski'
probability		True	max_features		None	metric_params		None
tol		0.001	random_state		12345	n_jobs		None
cache_size		200	max_leaf_nodes		None			
class_weight		None	min_impurity_decrease		0.0			
verbose		False	class_weight		None			
max_iter		-1	ccp_alpha		0.0			
decision_function_shape		'ovr'	monotonic_cst		None			
break_ties		False						
random_state		None						

En esta etapa , se utiliza un **StackingClassifier** que combina tres algoritmos diferentes para aumentar la variedad del aprendizaje:

- **SVM (RBF)**: está diseñado para manejar límites complicados. Necesita **datos muy limpios y normalizados**, ya que es muy sensible a las anomalías.
- **Árbol de Decisión**: Ofrece claridad y encuentra conexiones jerárquicas entre las variables.
- **KNN**: Clasifica basándose en la cercanía y la vecindad de los datos.

A diferencia del **Bagging**, que únicamente disminuye la varianza a través de SVMs, el **Stacking** también disminuye el sesgo. Al integrar la perspectiva geométrica (**SVM**), la lógica (**Árbol**) y la estadística (**KNN**), el modelo capta la información desde tres enfoques distintos.

El meta-modelo final utiliza una **Regresión Logística** diseñada para funcionar a través de los siguientes parámetros fundamentales:

- **Regularización Fuerte (C=0.1):** Al elegir un valor bajo para C, se aplica una penalización **L2 (Ridge)** que impide que el sistema se base solamente en el modelo más potente. Esto requiere que los tres estimadores lleguen a un acuerdo, distribuyendo los pesos y mejorando la habilidad de generalización.
- **Algoritmo de Optimización ("lbfgs"):** Este método se ha seleccionado por su elevada eficiencia en el uso de los recursos y su bajo requerimiento de memoria, lo que asegura una rápida convergencia hacia la solución ideal.
- **Ajuste del Intercepto (fit_intercept=True):** Esto permite que el modelo tenga un margen de maniobra, que es crucial para ajustar la frontera de decisión y adaptarse mejor a la realidad de los datos.

final_estimator		
LogisticRegression		
Parameters		
penalty		'l2'
dual		False
tol		0.0001
C		0.1
fit_intercept		True
intercept_scaling		1
class_weight		None
random_state		12345
solver		'lbfgs'
max_iter		1000
multi_class		'deprecated'
verbose		0
warm_start		False
n_jobs		None
l1_ratio		None

Obtenemos los mejores parámetros, la precisión y se realiza una evaluación en el conjunto de test.

Código:

```
print("Mejores parámetros:", grid_stack.best_params_)
print("Precisión en CV: {:.4f}".format(grid_stack.best_score_))
y_pred_stack = grid_stack.predict(X_test_sc)
print("\nInforme de Clasificación Stacking:")
print(classification_report(y_test, y_pred_stack))
```

```
Mejores parámetros: {'final_estimator__C': 0.1, 'passthrough': True}
Precisión en CV: 0.9921
```

```
Informe de Clasificación Stacking:
              precision    recall  f1-score   support

0               0.99         1.00         1.00         920
1               1.00         0.99         1.00         920

accuracy               1.00         1.00         1.00        1840
macro avg              1.00         1.00         1.00        1840
weighted avg           1.00         1.00         1.00        1840
```

El parámetro **passthrough=True** permite que el meta-modelo también acceda directamente a los **datos escalados originales**. Este nexo ayuda al meta-estimador a corregir eventuales fallos de los modelos base .

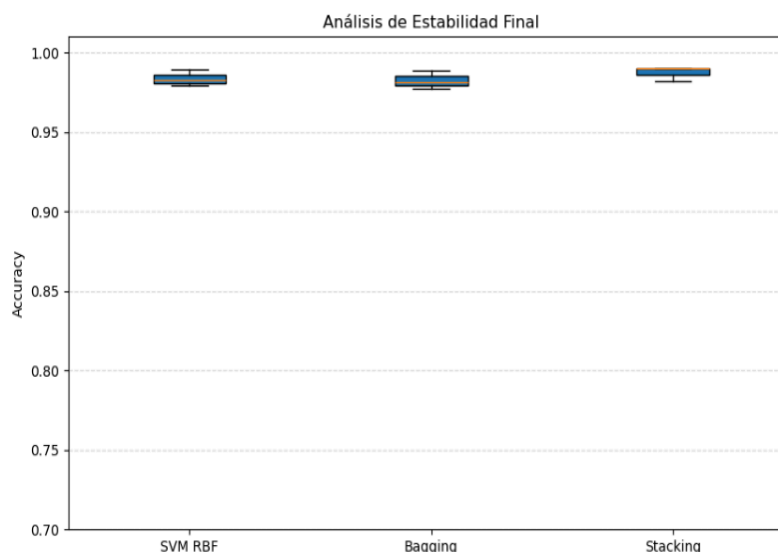
Como consecuencia, el **Stacking** muestra una efectividad notable, logrando una precisión del **99.21%** en validación cruzada. La combinación de una **regularización fuerte (C=0.1)** y la capacidad de **passthrough** asegura que el modelo final integre el conocimiento de varios algoritmos con la realidad de los datos crudos.

5. Comparación Global de Modelos

Para garantizar la robustez de la solución, se realizó una **Validación Cruzada (k=3)**, obteniendo 3 medidas de precisión independientes por modelo. Comparando la SVM RBF, el Bagging y el Stacking.

Código:

```
modelos_finales = {
    'SVM RBF': grid_rbf.best_estimator_,
    'Bagging': bagging_final,
    'Stacking': grid_stack.best_estimator_
}
resultados_estabilidad = {}
for nombre, modelo in modelos_finales.items():
    scores = cross_val_score(modelo, X_train_sc, y_train, cv=3, n_jobs=-1)
    resultados_estabilidad[nombre] = scores
    print(f"{nombre} - Media: {scores.mean():.4f}")
plt.figure(figsize=(10, 6))
plt.boxplot(resultados_estabilidad.values(), labels=resultados_estabilidad.keys(),
            patch_artist=True)
plt.title('Análisis de Estabilidad Final')
plt.ylabel('Accuracy')
plt.ylim(0.70, 1.01)
plt.grid(axis='y', linestyle='--', alpha=0.5) plt.show()
```



SVM RBF - Media: 0.9837
Bagging - Media: 0.9825
Stacking - Media: 0.9875

Se puede notar que el **Stacking** presenta el promedio más elevado (0.9875). En cuanto a la **estabilidad**, las tres cajas tienen una altura baja. Esto indica que, sin importar los datos que se utilicen, los modelos mantienen su consistencia.

● Discusión de Enfoques

SVM Lineal: Es conocido por su buena eficiencia en computación y su fácil interpretación, pero no puede captar relaciones que no son lineales, lo que lo hace inadecuado para este caso.

SVM RBF: Proporciona una capacidad predictiva más alta (99%) al reconocer límites complicados, sin embargo, su gran sensibilidad a los hiperparámetros requiere un ajuste cuidadoso para prevenir el sobreajuste.

Bagging: Proporciona alta solidez al disminuir la variabilidad al promediar 500 estimadores, asegurando resistencia al ruido, aunque implica un alto gasto de tiempo y memoria.

Stacking: Es el método más equilibrado. Su beneficio principal es su perspectiva multidisciplinaria (SVM, Árbol y KNN), donde un meta-clasificador evalúa las decisiones. A pesar de su complejidad técnica, representa la opción con mejor capacidad de generalización.

● Conclusión y Decisión Metodológica

La metodología ha **ido mejorando poco a poco**: comenzando con el modelo inicial (**Lineal**) y llegando al blindaje definitivo a través de la diversidad de algoritmos (**Stacking**).

El **Stacking se establece como el modelo exitoso** al abordar el problema desde tres enfoques matemáticos diferentes. En un ámbito empresarial como el churn, donde un error de falso negativo puede llevar a una pérdida económica directa, es crucial tener un modelo con la **menor varianza y la mayor estabilidad**.

El Stacking no solo mejora la precisión, sino que también proporciona la confianza necesaria para su implementación en un entorno real.